

PharmaMobile | Actividad autónoma S07

Estudiante: Guillen Mendoza Alexander

Docente: David Reyna

Sesión: 7 - Endpoints, DTO y pruebas de conexión

Fecha de ejecución: 22-23 de septiembre de 2026 (reloj del emulador en UTC)

Repositorio: https://github.com/alexgm-A11/PharmaMobile_Practica

Rama: feature/ktor-client

Commit del código: 4fcfdcc9407e69dee9b65d0ea5e29d1cc0f64fd5

1. Catálogo de endpoints

Recurso: productos. API pública de práctica Platzi, versión v1. URL base: <https://api.escuelajs.co/api/v1>. Los cinco métodos se ejecutaron el 22/09/2026; para POST/PUT/DELETE se creó y eliminó un producto temporal. La interfaz de esta sesión consume GET; el CRUD visual corresponde a la sesión 8.

Método y ruta	Parámetros	Respuesta comprobada	Errores a controlar
GET /products	limit, offset (query)	200, arreglo JSON	400, 500
GET /products/{id}	id positivo (ruta)	200, producto JSON	400 observado para ID inexistente; 404 posible
POST /products	JSON: title, price, description, categoryId, images	201, producto con id	400, 500
PUT /products/{id}	id + JSON completo	200, producto actualizado	400, 404, 500 (observado con cuerpo parcial)
DELETE /products/{id}	id (ruta)	200, true	400, 404, 500

La API pública cambia con el tiempo. Se comprobó GET de lista e ID 28 (200), POST de un producto temporal (201), PUT completo (200) y DELETE del mismo ID (200, true). Una petición PUT parcial devolvió 500; no se presenta como éxito. Un ID inexistente respondió 400, no el 404 propuesto como ejemplo en la guía.

Fuentes técnicas: documentación oficial de la API (<https://api.escuelajs.co/docs>) y documentación de Ktor (<https://ktor.io/docs/client-testing.html>).

2. Diccionario de DTO

Los tipos y valores por defecto coinciden con ProductoDto.kt. Un campo obligatorio no tiene valor por defecto. El producto remoto se convierte en ProductoCatalogo y se mantiene separado del Producto local porque la API no incluye stock.

ProductoDto

JSON	Tipo Kotlin	Obligatorio	Por defecto	Dominio
id	Int	Sí	-	ProductoCatalogo.id
title	String	Sí	-	ProductoCatalogo.nombre
price	Double	Sí	-	ProductoCatalogo.precio
description	String	No	""	ProductoCatalogo.descripcion
images	List<String>	No	emptyList()	ProductoCatalogo.imagen (primera)
category	CategoriaDto?	No	null	ProductoCatalogo.categoria (name)

CategoriaDto

JSON	Tipo Kotlin	Obligatorio	Por defecto	Dominio
id	Int	Sí	-	Sin equivalente; ID de API
name	String	Sí	-	ProductoCatalogo.categoria

Fragmento de JSON real de GET /products?limit=1&offset=0

```
[
  {
    "id":28,"title":"Sleek Modern Leather Sofa","price":60,
    "description":"Enhance the elegance of your living space...",
    "images":["https://i.imgur.com/Qphac99.jpeg", "..."],
    "category":{"id":3,"name":"Love is light", "slug":"love-is-light"},
    "slug":"sleek-modern-leather-sofa"}
]
```

El fragmento abrevia la descripción y el arreglo de imágenes; `slug` y las fechas sí llegaron en la respuesta real. La opción `ignoreUnknownKeys = true` permite ignorarlos sin alterar el DTO.

Código Kotlin de los DTO

```
@Serializable
data class CategoriaDto(val id: Int, val name: String)
```

```
@Serializable
data class ProductoDto(
    val id: Int,
    val title: String,
    val price: Double,
    val description: String = "",
    val images: List<String> = emptyList(),
    val category: CategoriaDto? = null
)
```

3. Bitácora | 01 - Respuesta exitosa

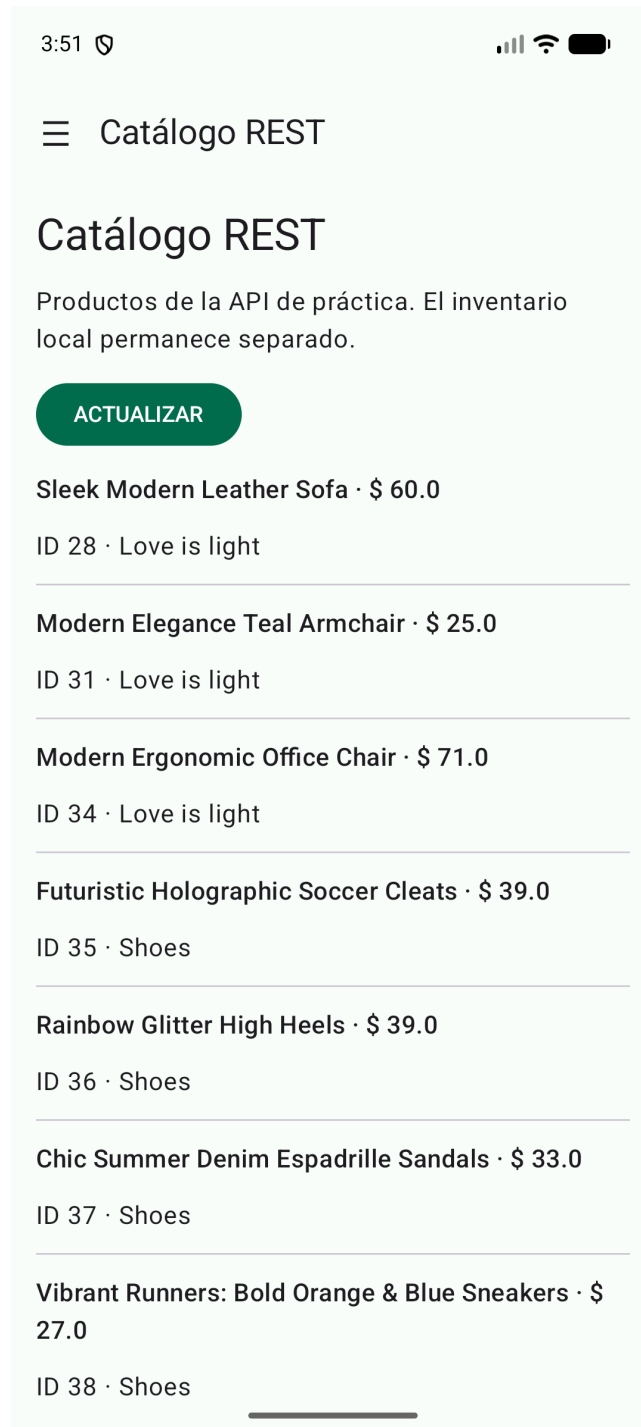
Fecha y plataforma: 23/09/2026 03:51 UTC; emulador Android Pixel 6.

Pasos: Abrir Catálogo REST con Internet y ejecutar GET /products?limit=10&offset=0.

Esperado: HTTP 200 y lista renderizada.

Observado: HTTP 200 en Logcat; se dibujaron nombres, precios, IDs y categorías.

Conclusión: La carga se completa y el usuario ve los productos.



Captura real: docs/capturas_s07/01-exito-android.png

3. Bitácora | 02 - Recurso inexistente

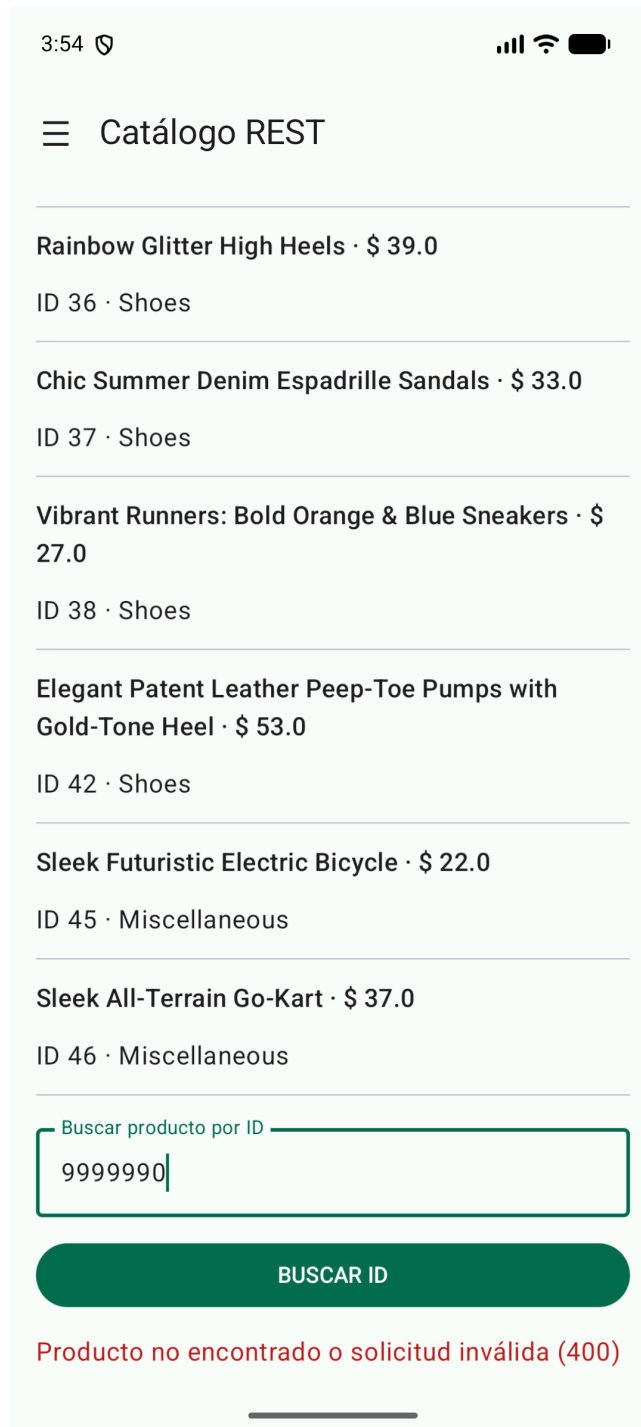
Fecha y plataforma: 23/09/2026 03:54 UTC; emulador Android Pixel 6.

Pasos: Buscar ID 9999990 desde el campo de la pantalla.

Esperado: Error HTTP controlado; la guía usa 404 como ejemplo.

Observado: La API respondió HTTP 400. La pantalla mostró 'Producto no encontrado o solicitud inválida (400)'.

Conclusión: El usuario recibe un mensaje y la aplicación permanece abierta.



Captura real: docs/capturas_s07/02-inexistente-android.png

3. Bitácora | 03 - Sin conexión

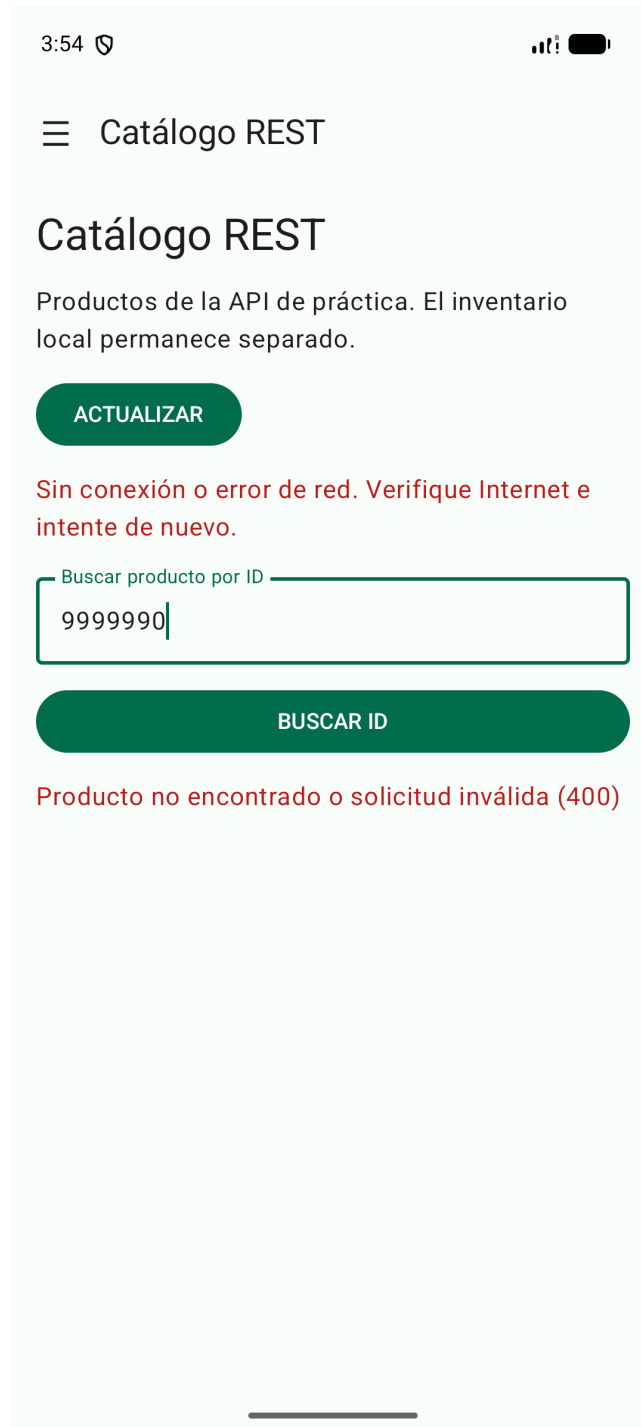
Fecha y plataforma: 23/09/2026 03:54 UTC; emulador Android Pixel 6.

Pasos: Desactivar Wi-Fi y datos del emulador; tocar ACTUALIZAR; restaurar ambos después.

Esperado: Excepción de red capturada, sin cierre de app.

Observado: La pantalla mostró 'Sin conexión o error de red. Verifique Internet e intente de nuevo.'.

Conclusión: La lista se sustituye por un error legible con opción de reintentar.



Captura real: docs/capturas_s07/03-sin-conexion-android.png

3. Bitácora | 04 - Tiempo de espera

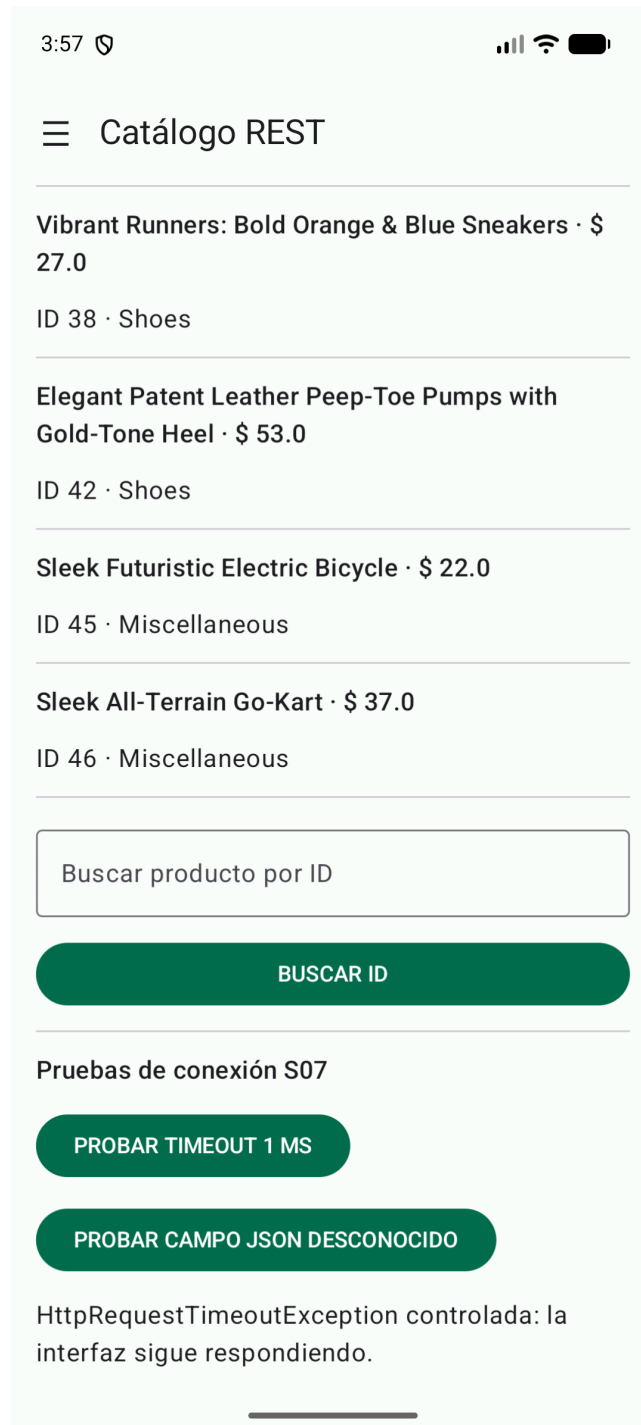
Fecha y plataforma: 23/09/2026 03:57 UTC; emulador Android Pixel 6.

Pasos: Con Internet, tocar PROBAR TIMEOUT 1 MS.

Esperado: HttpRequestTimeoutException controlada y UI reactiva.

Observado: La pantalla mostró 'HttpRequestTimeoutException controlada: la interfaz sigue respondiendo.'.

Conclusión: El usuario no queda bloqueado; la prueba no modifica datos.



Captura real: docs/capturas_s07/04-timeout-android.png

3. Bitácora | 05 - Campo JSON desconocido

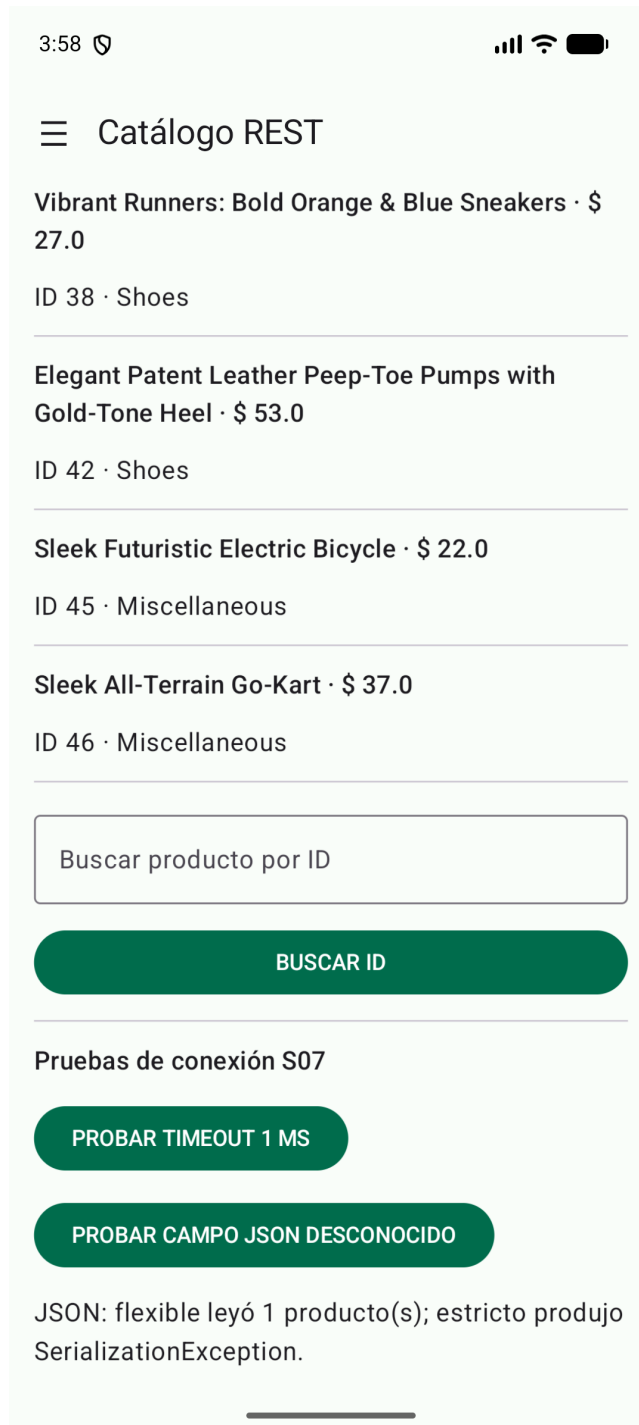
Fecha y plataforma: 23/09/2026 03:57 UTC; emulador Android Pixel 6.

Pasos: Con Internet, tocar PROBAR CAMPO JSON DESCONOCIDO.

Esperado: Modo flexible continúa; modo estricto lanza `SerializationException`.

Observado: Con el JSON real, el modo flexible leyó 1 producto y el estricto produjo `SerializationException`.

Conclusión: La app continúa; el resultado de diagnóstico explica la diferencia.



Captura real: docs/capturas_s07/05-json-desconocido-android.png

4. Evidencias, código y límites

Registro Ktor obtenido de Logcat del emulador Android

El cliente usa Logging con nivel ALL. A continuación se transcriben líneas representativas de una petición real; el cuerpo íntegro de la respuesta permanece consultable en Logcat. Este bloque es una transcripción, no una fotografía del panel Logcat.

```
09-23 03:57:47.934 I System.out: REQUEST: https://api.escuelajs.co/api/v1/products?limit=1&offset=0
09-23 03:57:49.349 I System.out: RESPONSE: 200 OK
09-23 03:57:49.349 I System.out: BODY Content-Type: application/json; charset=utf-8
09-23 03:57:49.349 I System.out: BODY START
09-23 03:57:49.349 I System.out: [{"id":28,"title":"Sleek Modern Leather Sofa",...}]
09-23 03:57:49.349 I System.out: BODY END
```

Verificación automatizada y estructura

`ProductoApiTest` contiene cinco pruebas con MockEngine: 200, recurso inexistente, red ausente, timeout y JSON desconocido. `:shared:testAndroidHostTest`, `:androidApp:assembleDebug` y `:shared:compileKotlinIosSimulatorArm64` terminaron correctamente. `ProductoApi.kt` configura cliente Ktor; `ProductoDto.kt` declara los DTO; `ProductoCatalogoRepositorioRest.kt` convierte a dominio; `CatalogoViewModel.kt` maneja estado y mensajes; `CatalogoScreen.kt` dibuja la interfaz.

Evidencia iOS pendiente

El código compartido compiló para iOS Simulator ARM64, pero esta máquina usa Windows y no puede ejecutar el simulador iOS. No se incluye una captura iOS inexistente. Debe abrirse iosApp en Xcode sobre macOS y capturarse la pantalla Catálogo REST conectada antes de marcar este punto como completo.

Repositorio y commit

Repositorio: https://github.com/alexgm-A11/PharmaMobile_Practica

Rama individual: feature/ktor-client

Commit de la implementación documentada:

https://github.com/alexgm-A11/PharmaMobile_Practica/commit/4fcfdcc9407e69dee9b65d0ea5e29d1cc0f64fd5

README.md incluye la sección Conectividad REST.

Estado honesto de la lista de cotejo

Completos: catálogo de cinco endpoints ejecutados; diccionario de ambos DTO; JSON y código; cinco escenarios con capturas Android; README y commit. Pendientes: captura iOS y fotografía específica del panel Logcat (se incluye la transcripción real).